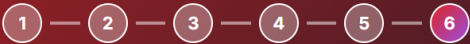


Learning Week: Day 1 → Day 6

A complete guide to understanding chains, agents, memory, orchestration, and building a real retail data pipeline



Day 1: Chains

Day 2: Templates

Day 3: Memory & Tools

Day 4: LangGraph

Day 5: Orchestrator

Day 6: The Project

Day 6: The Project – Retail Data Pipeline

4 Agents · Real CSV Data · End-to-End Automation

What We're Building

A multi-agent LangGraph pipeline that takes a raw retail CSV file and automatically cleans it, analyses it, and produces a dashboard-ready summary — all coordinated by an orchestrator agent.

Input: Retail CSV

Output: Insights + Charts

LangChain + LangGraph

The 4-Agent Pipeline



Agent Specifications

01

Orchestrator Agent

Manager · Router · Coordinator

The orchestrator is the entry point. It receives the raw retail CSV, inspects it, and decides the correct processing sequence — always cleaning before analysing. It coordinates all other agents and collects their outputs into a final report.

Think of it as the **project manager**: it never does the actual work itself, it just makes sure the right agent handles each job in the right order.

RESPONSIBILITIES

Receive: Raw retail CSV as input

Inspect: Understand what the data contains

Decide: Set processing order (Clean → Analyse → Visualise)

Route: Pass data to the correct agent at each step

Collect: Gather outputs from all agents

Return: Final aggregated result to the user

ORCHESTRATOR LOGIC (PSEUDO-CODE):

```
def orchestrator(csv_path):
    state = {"raw_data": load_csv(csv_path)}

    # Always clean first
```

```
state = clean_agent.run(state)

# Then analyse the clean data
state = analysis_agent.run(state)

# Bonus: visualise if requested
if state.get("visualize"):
    state = viz_agent.run(state)

return state["final_report"]
```

02

Clean Agent

Data Cleaner · Validator · Transformer

Raw retail data is almost always messy — missing prices, wrong date formats, duplicate transactions, invalid category names. The Clean Agent handles all of this before any analysis happens. Garbage in, garbage out — this agent prevents that.

It receives the raw DataFrame from the orchestrator and outputs a **cleaned, validated dataset** ready for analysis.

RESPONSIBILITIES

Missing Values: Fill or drop null entries in key columns

Data Types: Convert dates, prices, quantities to correct types

Duplicates: Remove identical or near-duplicate rows

Broken Rows: Drop rows with invalid/impossible values

Standardise: Normalise category names and region labels

Output: Return a clean DataFrame to the orchestrator

WHAT THE CLEAN AGENT FIXES:

```
# Before cleaning (raw CSV issues):
# - "price" column has "$" symbols → not numeric
# - "date" is stored as "01/15/2024" string
# - 3% of rows have NaN in "quantity"
# - 12 duplicate transaction rows
# - "Electronics", "electronics", "ELECTRONICS" → inconsistent

# After Clean Agent:
# - price: float64 ✓
# - date: datetime64 ✓
# - quantity NaNs: filled with median ✓
# - duplicates: removed ✓
# - categories: lowercase, standardised ✓
```

03

Analysis (EDA) Agent

Data Analyst · Insight Generator · Summariser

Once the data is clean, the Analysis Agent runs Exploratory Data Analysis (EDA) — it digs into the numbers and extracts the key business insights. It doesn't just compute statistics; it produces a structured, readable summary.

The output is a **structured insights object** (dictionary/JSON) that can feed a dashboard or be displayed directly to the user.

INSIGHTS GENERATED

Total Sales: Sum of revenue across all transactions

Best-Selling Products: Top N products by quantity and revenue

Sales by Region: Breakdown of revenue per geographic region

Sales by Category: Revenue split across product categories

Time Trends: Monthly or weekly sales trends

Summary Stats: Avg order value, total transactions, peak period

ANALYSIS AGENT OUTPUT (STRUCTURED):

```
{
  "total_revenue": 847293.50,
  "total_transactions": 12841,
  "avg_order_value": 65.98,
  "top_products": [
    {"name": "Wireless Headphones", "revenue": 92400},
    {"name": "Laptop Stand", "revenue": 78200}
  ],
  "sales_by_region": {
    "North": 312000, "South": 198000,
    "East": 201000, "West": 136293
  },
  "sales_by_category": {
```

```
"Electronics": 420000,
"Accessories": 230000,
"Stationery": 197293
},
"peak_month": "November 2024"
}
```

04

Visualization Agent BONUS

Chart Builder - Dashboard Generator

After the Analysis Agent produces structured insights, the Visualization Agent turns that data into charts. It only runs once the first three agents are working reliably — treat it as a stretch goal.

It reads the analysis output and generates **bar charts**, **line graphs**, and **pie charts** using a library like Matplotlib or Plotly, saving them as image files or rendering them inline.

CHARTS TO GENERATE

Bar Chart: Top products by revenue

Pie Chart: Sales split by category

Bar Chart: Revenue by region

Line Chart: Monthly sales trend over time

Output: Save as PNG files or display in notebook

VISUALIZATION AGENT (PSEUDO-CODE):

```
def viz_agent(state):
    insights = state["insights"]

    # Bar chart: top products
    plot_bar(insights["top_products"], title="Top Products by Revenue")

    # Pie chart: sales by category
    plot_pie(insights["sales_by_category"], title="Revenue by Category")

    # Line chart: monthly trend
    plot_line(insights["monthly_trend"], title="Sales Over Time")

    state["charts_saved"] = True
    return state
```

Recommended Build Order

1 Clean Agent first

Test it standalone with a sample CSV. Verify output is clean before touching anything else.

2 Analysis Agent second

Feed it the clean output from step 1. Check all insight fields are correct.

3 Orchestrator Agent third

Wire agents 1 and 2 together. Test the full pipeline end-to-end with the raw CSV.

4 Visualization Agent last (bonus)

Only add this once the first 3 agents work reliably end-to-end.

Key Takeaway: Build and test each agent in isolation before connecting them. The orchestrator is the last piece to wire up, not the first. This way, if something breaks, you know exactly which agent is responsible.

Free Resources to Dive Deeper

LangChain Docs

Official documentation with API reference and guides

LangChain Academy

Free official courses on LangChain fundamentals

LangGraph Docs

Guide to building graphs and agents

Multi-Agent Tutorial

Real example of orchestrator agents in action

🔴 Learning Week: LangChain + LangGraph Fundamentals

DevSynt AI Automation Internship · Mentor: Afnan Shoukat

Quiz coming next — make sure you understand these concepts!